

HCL AppScan Report

Report Type: Security Report

Scan Name: DAST 2026-09-10 <https://aaca.ntua.edu.tw/Server>

Technology: DAST

Report created at: Tuesday, September 15, 2026

This report was generated by HCL AppScan v2.1

Summary of security issues

Medium severity issues:	1
Low severity issues:	3
Informational severity issues:	10
Total security issues:	14

Scan Information

Scan started: Monday, September 14, 2026 1:28:23 AM (UTC)

Test policy: Default

Test Optimization: No Optimization

Hosts

Host: aaca.ntua.edu.tw
Port: 443
Operating system: Unknown
Web server: Unknown
Application Server: Any

Login Settings

Login method: Recorded login
Concurrent Logins: Enabled
In-session detection: Enabled
In-session pattern: UserId:".admin02|UserName:".admin02
Tracked or session ID cookies:

Tracked or session ID parameters: ->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
enumName
enumName
enumName
enumName
->"Fields"[3]
->"Fields"[1]
->"Fields"[3]
->"Fields"[1]
->"Fields"[1]
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Condition"
->"Fields"[1]
->"Fields"[3]
->"Condition"
enumName

Login sequence: https://aaca.ntua.edu.tw/Server/Login
http://efaidnbmnnnibpcajpcgclclefindmkaj/dist/FloatingActionButton.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/common/local-storage.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/content_scripts/utills/utill.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/content_scripts/utills/fte-utills.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/content_scripts/express/express-fte.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/content_scripts/express/express-utills.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/dist/extensionShared.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/dist/vendor.js
http://efaidnbmnnnibpcajpcgclclefindmkaj/dist/otherVendors.js
https://aaca.ntua.edu.tw/Service/SystemAPI/GetXsrftoken

https://aaca.ntua.edu.tw/Service/Auth/Login
https://aaca.ntua.edu.tw/Service/SystemAPI/GetXsrfToken
https://aaca.ntua.edu.tw/Service/Auth/Me
https://aaca.ntua.edu.tw/Service/Auth/Refresh
https://aaca.ntua.edu.tw/Service/SystemAPI/GetXsrfToken
https://aaca.ntua.edu.tw/Service/SpecHomePageApi/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/SpecHomePageApi/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/SpecHomePageApi/QueryList
https://aaca.ntua.edu.tw/Service/SpecHomePageApi/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/SpecHomePageApi/QueryData%3FinternalId=c2080823-a98b-4550-92d1-2fad373c5886?internalId=c2080823-a98b-4550-92d1-2fad373c5886
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Banner/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=WindowTarget?enumName=WindowTarget
https://aaca.ntua.edu.tw/Service/Banner/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Banner/GetTotalCounts
https://aaca.ntua.edu.tw/Service/Banner/QueryList
https://aaca.ntua.edu.tw/Service/Announcement/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ContentStatus?enumName=ContentStatus
https://aaca.ntua.edu.tw/Service/Announcement/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Announcement/GetTotalCounts
https://aaca.ntua.edu.tw/Service/Announcement/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Category/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/PageManagement/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/PageManagement/GetTotalCounts
https://aaca.ntua.edu.tw/Service/PageManagement/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Category/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Gallery/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ContentStatus?enumName=ContentStatus
https://aaca.ntua.edu.tw/Service/Gallery/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Gallery/GetTotalCounts
https://aaca.ntua.edu.tw/Service/Gallery/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/FileArchive/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ContentStatus?enumName=ContentStatus
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=WindowTarget?enumName=WindowTarget
https://aaca.ntua.edu.tw/Service/FileArchive/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/FileArchive/GetTotalCounts
https://aaca.ntua.edu.tw/Service/FileArchive/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Account/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Account/GetTotalCounts
https://aaca.ntua.edu.tw/Service/Account/QueryList
https://aaca.ntua.edu.tw/Service/RolePermission/QueryList
https://aaca.ntua.edu.tw/Service/SiteMenu/QueryList
https://aaca.ntua.edu.tw/Service/SiteMenu/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=WindowTarget?enumName=WindowTarget
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=MenuUrlType?enumName=MenuUrlType
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ModulePageType?enumName=ModulePageType

https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ModuleDisplayStyle?enumName=ModuleDisplayStyle
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/PageManagement/QueryList
https://aaca.ntua.edu.tw/Service/Banner/QueryList
https://aaca.ntua.edu.tw/Service/Timeline/QueryList
https://aaca.ntua.edu.tw/Service/Survey/QueryList
https://aaca.ntua.edu.tw/Service/SiteMenu/QueryData%3FinternalId=e27fdc50-9e19-4067-a593-c9e3354366ee?internalId=e27fdc50-9e19-4067-a593-c9e3354366ee
https://aaca.ntua.edu.tw/Service/Announcement/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Announcement/GetTotalCounts
https://aaca.ntua.edu.tw/Service/Announcement/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/SiteMenu/QueryList
https://aaca.ntua.edu.tw/Service/SiteMenu/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=WindowTarget?enumName=WindowTarget
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=MenuUrlType?enumName=MenuUrlType
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ModulePageType?enumName=ModulePageType
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ModuleDisplayStyle?enumName=ModuleDisplayStyle
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/PageManagement/QueryList
https://aaca.ntua.edu.tw/Service/Banner/QueryList
https://aaca.ntua.edu.tw/Service/Timeline/QueryList
https://aaca.ntua.edu.tw/Service/Survey/QueryList
https://aaca.ntua.edu.tw/Service/SiteMenu/QueryData%3FinternalId=e27fdc50-9e19-4067-a593-c9e3354366ee?internalId=e27fdc50-9e19-4067-a593-c9e3354366ee
https://aaca.ntua.edu.tw/Service/Announcement/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Announcement/GetTotalCounts
https://aaca.ntua.edu.tw/Service/Announcement/QueryList
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/Announcement/GetModelDisplayName
https://aaca.ntua.edu.tw/Service/Category/QueryList
https://aaca.ntua.edu.tw/Service/Tag/QueryList
https://aaca.ntua.edu.tw/Service/SystemAPI/GetEnumOptions%3FenumName=ContentStatus?enumName=ContentStatus

Table of Contents

Summary

- Issues

Issue Types

- Cookie with Insecure or Improper or Missing SameSite attribute
- Trusted types in scripts not enforced in CSP
- Application Error
- Client-Side (JavaScript) Cookie References
- Email Address Pattern Found

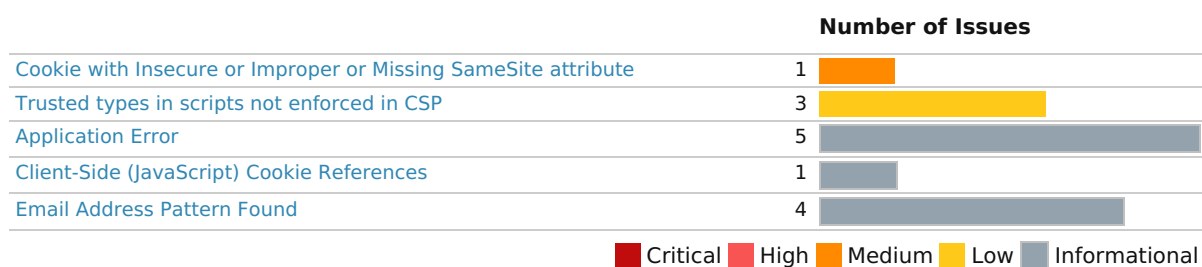
How to Fix

- Application Error
- Client-Side (JavaScript) Cookie References
- Cookie with Insecure or Improper or Missing SameSite attribute
- Email Address Pattern Found
- Trusted types in scripts not enforced in CSP

Summary

Total security issues: **14**

Issue Types: **5**



Issues - By Issue Types:

How to Fix: Cookie with Insecure or Improper or Missing SameSite attribute

Issue 1 of 1

Issue ID:	4ffd433e-f36b-1410-89cd-00f37f576f24
Severity:	Medium
Status	Open
Location	https://aaca.ntua.edu.tw/
Domain	aaca.ntua.edu.tw
Element	wcms.visitor (Cookie)
Path	/
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	4.7
Date Created	Thursday, August 27, 2026
Last Updated	Thursday, August 27, 2026
CWE:	1275

Issue 1 of 1 - Details

Reasoning: The response contains Sensitive Cookie with Insecure or Improper or Missing SameSite attribute, which may lead to Cookie information leakage, which may extend to Cross-Site-Request-Forgery(CSRF) attacks if there are no additional protections in place.

Test Requests and Responses:

[illegible]

cross-origin-opener-policy: same-origin
cross-origin-embedder-policy: require-corp
cross-origin-resource-policy: same-origin
x-permitted-cross-domain-policies: none
Content-Security-Policy: default-src 'none'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'none'
Strict-Transport-Security: max-age=16070400; includeSubDomains
X-Powered-By:
Date: Mon, 14 Sep 2026 02:16:25 GMT
Content-Type: text/html
Expires: 0
Content-Length: 1215

[Go to Table of Contents](#)

L DAST: Trusted types in scripts not enforced in CSP 3

How to Fix: [Trusted types in scripts not enforced in CSP](#)

Issue 1 of 3

Issue ID:	6547443e-f36b-1410-89cd-00f37f576f24
Severity:	Low
Status	Open
Location	https://aaca.ntua.edu.tw/service/filemanagement/public_preview/b8233f19-730a-4f8d-8005-4c47f9ebf7b1
Domain	aaca.ntua.edu.tw
Element	b8233f19-730a-4f8d-8005-4c47f9ebf7b1 (Page)
Path	/service/filemanagement/public_preview/b8233f19-730a-4f8d-8005-4c47f9ebf7b1
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	3.7
Date Created	Thursday, September 10, 2026
Last Updated	Thursday, September 10, 2026
CWE:	1032

Issue 1 of 3 - Details

Reasoning: Missing require-trusted-types-for 'script'

Test Requests and Responses:

這個要求/回應包含二進位內容，而產生的報告中並未包含二進位內容。

Issue 2 of 3

Issue ID:	2e4e443e-f36b-1410-89cd-00f37f576f24
Severity:	Low
Status	Open
Location	https://aaca.ntua.edu.tw/service/filemanagement/public_download/34ca86b5-586e-4c6e-89f1-dc66f8a5e99b
Domain	aaca.ntua.edu.tw
Element	34ca86b5-586e-4c6e-89f1-dc66f8a5e99b (Page)
Path	/service/filemanagement/public_download/34ca86b5-586e-4c6e-89f1-dc66f8a5e99b
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	3.7
Date Created	Thursday, September 10, 2026
Last Updated	Thursday, September 10, 2026
CWE:	1032

Issue 2 of 3 - Details

Reasoning: Missing require-trusted-types-for 'script'

Test Requests and Responses:

這個要求/回應包含二進位內容，而產生的報告中並未包含二進位內容。

Issue 3 of 3

Issue ID:	19fd433e-f36b-1410-89cd-00f37f576f24
Severity:	Low
Status	Open
Location	https://aaca.ntua.edu.tw/service/systemapi/getxsrfToken
Domain	aaca.ntua.edu.tw
Element	GetXsrfToken (Page)
Path	/service/systemapi/getxsrfToken
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	3.7
Date Created	Thursday, August 27, 2026
Last Updated	Thursday, August 27, 2026
CWE:	1032

Issue 3 of 3 - Details

Reasoning: Missing require-trusted-types-for 'script'

Test Requests and Responses:

```
GET /Service/SystemAPI/GetXsrftoken HTTP/1.1
Host: aaca.ntua.edu.tw
Connection: keep-alive
sec-ch-ua-platform: "Linux"
X-XSRF-TOKEN: CfdJ8HckeMFvsBBAitncu7Dg82y-O_3iR34ZjCvYk9wUIVcavizj8MrPmGP8xq8nnCoXgKK_hYR5wUjV6V30kNcwD7-
DwZvyGwR7f4HbMBG0OLpExZSLZPtotfAJTWYPcZZjGFuB16v43nmTOOWP03Am0BNj7Mymn4zuqUY0xiiNjzhnrzVjh30nf_vXct2aC8qA
Accept-Language: en-US
Accept: application/json, text/plain, */*
sec-ch-ua: "Chromium";v="1145", "Not:A-Brand";v="99"
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/145.0.0.0 Safari/537.36
sec-ch-ua-mobile: ?0
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://aaca.ntua.edu.tw/
Accept-Encoding: gzip
Cookie: wcms.lang=zh-tw; __Host-WCMS-Antiforgery=CfdJ8HckeMFvsBBAitncu7Dg82yUciz_W6KLIM7wXh_u1LCixd8xoFmHLisUUIja51E-1oCouC0SDt8Cfhmqw5DvjEvSvB9a9ziHODJempfqXVLC--Eq19x0POWwKaMijAedoRLYYr8AtjdY2DpS7qLQXo;
access=eYjhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eYJbnRlcml5bElkljoiMmE1NzQ3Y2MtZTBjNy00MwZkLWFmYTltOTNiYUxNzc2NDY2liwiaHR0cDovL3NjaGVtYXNueG1sc29hcC5vcmcvd3MvMjAwNS8wNS9pZGVudGloes9jbGFpbXNvbWZlcnRlZCjldGkiOiJpbnNwZkYzAzOGNkZmEOMDAZyE3MzYwNDE3NTFjMTk5NiIsIm5iZi56cmctOTM0MT00TmTzNiwiZiXhwjloxNzg5MzUxMTM2LClpc3MiOiJXQ011tiwiYXVkljoiVONNUy5BZG1pbj9.mTAquR4PXRaGUb6lE2oMreP08ZCy4Nwwwr7fa0uqC
Pk; rtid=49976652abf8429fb1b4b35702f46acf; XSRF-TOKEN=CfdJ8HckeMFvsBBAitncu7Dg82y-O_3iR34ZjCvYk9wUIVcavizj8MrPmGP8xq8nnCoXgKK_hYR5wUjV6V30kNcwD7-
DwZvyGwR7f4HbMBG0OLpExZSLZPtotfAJTWYPcZZjGFuB16v43nmTOOWP03Am0BNj7Mymn4zuqUY0xiiNjzhnrzVjh30nf_vXct2aC8qA;
wcms.visitor=e2abdc5c7ed44cbf8a1ebbe44247a7022
Content-Length: 0

HTTP/1.1 204 No Content
Cache-Control: no-store, must-revalidate, proxy-revalidate, no-cache
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
permissions-policy: geolocation=(), microphone=(), camera=(), fullscreen=()
cross-origin-opener-policy: same-origin
cross-origin-embedder-policy: require-corp
cross-origin-resource-policy: same-origin
x-permitted-cross-domain-policies: none
Content-Security-Policy: default-src 'none'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'none'
Strict-Transport-Security: max-age=16070400; includeSubDomains
X-Powered-By:
Date: Mon, 14 Sep 2026 02:07:22 GMT
Expires: 0
Set-Cookie: XSRF-
TOKEN=CfdJ8HckeMFvsBBAitncu7Dg82x4jxOlqpgNnhuEWguVko7nlgBqeNrdaEjbr4LxgsGUFAMq_DQSRqfDL_OlpOOi2j_kXB2vtMper7x9znjKQHxz9vYctIPC
Yoz0SHVilUW0i0ueVAVoOsdneGoFFWOG8V9o; path=/; secure: sameSite=strict
```

[Go to Table of Contents](#)

Issue ID:	f0b0433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/service/systemapi/getenumoptions
Domain	aaca.ntua.edu.tw
Element	enumName (Parameter)
Path	/service/systemapi/getenumoptions
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	550

Difference: Parameter `enumName` manipulated from: `ContentStatus` to: `%00`

Test Requests and Responses:

```
HTTP/1.1 404 Not Found
Cache-Control: no-store, must-revalidate, proxy-revalidate, no-cache
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
permissions-policy: geolocation=(), camera=(), fullscreen=(self)
cross-origin-opener-policy: same-origin
cross-origin-embedder-policy: require-corp
cross-origin-resource-policy: same-origin
x-permitted-cross-domain-policies: none
Content-Security-Policy: default-src 'none'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'none'
Strict-Transport-Security: max-age=16070400; includeSubDomains
X-Powered-By:
Date: Mon, 14 Sep 2026 04:55:23 GMT
Content-Type: text/html
Expires: 0
Content-Length: 1229
```

9

Issue 2 of 5

Issue 2 of 5 - Details

Test Requests and Responses:

10

```
<style type="text/css">
<!--
body{margin:0;font-size:.7em;font-family:Verdana, Arial, Helvetica, sans-serif;background:#EEEEEE;}
fieldset{padding:0 15px 10px 15px;}
h1{font-size:2.4em;margin:0;color:#FFF;}
h2{font-size:1.7em;margin:0;color:#CC0000;}
h3{font-size:1.2em;margin:10px 0 0 0;color:#000000;}
#header{width:96%;margin:0 0 0 0;padding:6px 2% 6px 2%;font-family:"trebuchet MS", Verdana, sans-serif;color:#FFF;
background-color:#555555;}
#content{margin:0 0 0 2%;position:relative;}
.content-container{background:#FFF;width:96%;margin-top:9px;padding:10px;position:relative;}
-->
</style>
</head>
<body>
<div id="header"><h1>Server Error</h1></div>
<div id="content">
<div class="content-container"><fieldset>
<h2>404 - File or directory not found.</h2>
<h3>The resource you are looking for might have been removed, had its name changed, or is temporarily unavailable.</h3>
</fieldset></div>
</div>
</body>
</html>
```

Issue 3 of 5

Issue ID:	c5ad433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/service/filemanagement/public_preview/b8233f19-730a-4f8d-8005-4c47f9ebf7b1
Domain	aaca.ntua.edu.tw
Element	[GUID] (Parameter)
Path	/service/filemanagement/public_preview/b8233f19-730a-4f8d-8005-4c47f9ebf7b1
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	550

Issue 3 of 5 - Details

Difference: Parameter manipulated from: b8233f19-730a-4f8d-8005-4c47f9ebf7b1 to:

Reasoning: The application has responded with an error message, indicating an undefined state that may expose sensitive information.

Test Requests and Responses:

```
GET /Service/FileManagement/Public_Preview/;?
fileName=116%E5%AD%B8%E5%B9%B4%E5%BA%A6%E7%A0%94%E7%A9%B6%E6%B9%80%E5%90%84%E9%A0%85%E5%85%A5%E5%AD%B8%E
6%8B%9B%E7%94%9F%E8%80%83%E8%A9%A6%E9%A0%90%E5%AE%9A%E6%97%A5%E7%A8%8B%E8%A1%A8%28%E5%83%85%E4%BE%9B%E
%8F%83%E8%80%83%29 HTTP/1.1
Accept-Language: en-US
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Referer: https://aaca.ntua.edu.tw/
Host: aaca.ntua.edu.tw
User-Agent: Mozilla/5.0 (Windows NT 6.2; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.127 Safari/537.36
Cookie: __Host-WCMS-Antiforgery=CfDj8HckemFvsBBaitncu7Dg82yUciz_W6KLIM7wXh_u1LCxidd8xoFmHLisUuija51E-
1oCouC0SDt8Cfhmqw5DvjEvsVbEaE9dnjODjempfqXVLC--Eq19x0POWRKaMijAedoRLYYr8AtjdY2Dp57qLQXo; wcms.lang=zh-tw;
access=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJjbGRLcm5hbElkIjoImE1NzQ3Y2MtZTBjNy00MWZkLWwFmYtITOTNiYzUxNzczNDY2IiwiaHR0cDovL3NjaGVt
YXMuG1sc29hcC5vcmcvd3MvMjAwNS8wNS9pZGVudG0eS9jbGFpXWVmbmFtZSI6ImFkbWwudWUudmllClJqdGkiOiJlNWZkYzAzOGNkZmE0MDAzYjE3MzYwNDE3NTFjMTk5NiIsIm5iZ
i6MTc0OTM0OTMzMzNiZWZwhlJpYXNzZmZUxMTM2LClJpc3MiOiJXQ01TliwiYXVkljoiVONNUy5BZG1pbjJ9.MtAqur4PXRaGub6lE2oMreP08ZCy4Nwwmr7fa0uqc
Pk; rtid=49976652abf8429fb1b4b35702f46acf; XSrf-TOKEN=CfDj8HckemFvsBBaitncu7Dg82xhrrpdRhZa-
X365RBN8x4OPuxR6p9f_wf_VrKj_79a0JAErnXcFTP-
YBsh75796mpjPb0CxiwftfqWMEYumjBZu95q_w3o9SU7vYAEs7tQbR2to9ZYcojzZ2A53kHlokbiQUOoPSlXoYt8YxPu3V5jK3Ma5_5qzwKVvsOpvjw
Content-Length: 0

HTTP/1.1 404 Not Found
Cache-Control: no-store, must-revalidate, proxy-revalidate, no-cache
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
permissions-policy: geolocation=(), camera=(), fullscreen=(self)
cross-origin-opener-policy: same-origin
cross-origin-embedder-policy: require-corp
cross-origin-resource-policy: same-origin
x-permitted-cross-domain-policies: none
Content-Security-Policy: default-src 'none'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'none'
Strict-Transport-Security: max-age=16070400; includeSubDomains
X-Powered-By:
Date: Mon, 14 Sep 2026 04:55:23 GMT
Content-Type: text/html
Expires: 0
Content-Length: 1229

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head>
<meta http-equiv="Content-Type" content="text/html; charset=big5"/>
<title>404 - 找不到檔案或目錄。</title>
<style type="text/css">
<!--
body{margin:0;font-size:.7em;font-family:Verdana, Arial, Helvetica, sans-serif;background:#EEEEEE;}
fieldset{padding:0 15px 10px 15px;}
h1{font-size:2.4em;margin:0;color:#FFF;}
h2{font-size:1.7em;margin:0;color:#CC0000;}
h3{font-size:1.2em;margin:10px 0 0 0;color:#000000;}
#header{width:96%;margin:0 0 0 0;padding:6px 2% 6px 2%;font-family:"trebuchet MS", Verdana, sans-serif;color:#FFF;
background-color:#555555;}
#content{margin:0 0 0 2%;position:relative;}
.content-container{background:#FFF;width:96%;margin-top:9px;padding:10px;position:relative;}
-->
</style>
</head>
<body>
<div id="header"><h1>Server Error</h1></div>
<div id="content">
<div class="content-container"><fieldset>
<h2>404 - File or directory not found.</h2>
<h3>The resource you are looking for might have been removed, had its name changed, or is temporarily unavailable.</h3>
</fieldset></div>
</div>
</body>
</html>
```

Issue 4 of 5

Issue ID:	d4ad433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/service/filemanagement/public_download/4a9ae5bc-7f54-48fb-9a44-4fbb91ed11bc
Domain	aaca.ntua.edu.tw
Element	[GUID] (Parameter)
Path	/service/filemanagement/public_download/4a9ae5bc-7f54-48fb-9a44-4fbb91ed11bc
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	550

Issue 4 of 5 - Details

Difference: Parameter manipulated from: 4a9ae5bc-7f54-48fb-9a44-4fbb91ed11bc to:)

Reasoning: The application has responded with an error message, indicating an undefined state that may expose sensitive information.

Test Requests and Responses:

[illegible]

```
<head>
<meta http-equiv="Content-Type" content="text/html; charset=big5"/>
<title>404 - 找不到檔案或目錄。 </title>
<style type="text/css">
<!--
body{margin:0;font-size:.7em;font-family:Verdana, Arial, Helvetica, sans-serif;background:#EEEEEE;}
fieldset{padding:0 15px 10px 15px;}
h1{font-size:2.4em;margin:0;color:#FFF;}
h2{font-size:1.7em;margin:0;color:#CC0000;}
h3{font-size:1.2em;margin:10px 0 0 0;color:#000000;}
#header{width:96%;margin:0 0 0 0;padding:6px 2% 6px 2%;font-family:"trebuchet MS", Verdana, sans-serif;color:#FFF;
background-color:#555555;}
#content{margin:0 0 2%;position:relative;}
.content-container{background:#FFF;width:96%;margin-top:9px;padding:10px;position:relative;}
-->
</style>
</head>
<body>
<div id="header"><h1>Server Error</h1></div>
<div id="content">
<div class="content-container"><fieldset>
<h2>404 - File or directory not found.</h2>
<h3>The resource you are looking for might have been removed, had its name changed, or is temporarily unavailable.</h3>
</fieldset></div>
</div>
</body>
</html>
```

Issue 5 of 5

Issue ID:	69af433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/service/filemanagement/public_preview/5d753178-af87-4d57-8c36-26221e33215f
Domain	aaca.ntua.edu.tw
Element	[GUID] (Parameter)
Path	/service/filemanagement/public_preview/5d753178-af87-4d57-8c36-26221e33215f
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	550

Issue 5 of 5 - Details

Difference: Parameter -- manipulated from: 5d753178-af87-4d57-8c36-26221e33215f to: 

Reasoning: The application has responded with an error message, indicating an undefined state that may expose sensitive information.

Test Requests and Responses:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
```

```

<head>
<meta http-equiv="Content-Type" content="text/html; charset=big5"/>
<title>404 - 找不到檔案或目錄。</title>
<style type="text/css">
<!--
body{margin:0;font-size:.7em;font-family:Verdana, Arial, Helvetica, sans-serif;background:#EEEEEE;}
fieldset{padding:0 15px 10px 15px;}
h1{font-size:2.4em;margin:0;color:#FFF;}
h2{font-size:1.7em;margin:0;color:#CC0000;}
h3{font-size:1.2em;margin:10px 0 0 0;color:#000000;}
#header{width:96%;margin:0 0 0 0;padding:6px 2% 6px 2%;font-family:"trebuchet MS", Verdana, sans-serif;color:#FFF;
background-color:#555555;}
#content{margin:0 0 0 2%;position:relative;}
.content-container{background:#FFF;width:96%;margin-top:9px;padding:10px;position:relative;}
-->
</style>
</head>
<body>
<div id="header"><h1>Server Error</h1></div>
<div id="content">
<div class="content-container"><fieldset>
<h2>404 - File or directory not found.</h2>
<h3>The resource you are looking for might have been removed, had its name changed, or is temporarily unavailable.</h3>
</fieldset></div>
</div>
</body>
</html>

```

[Go to Table of Contents](#)

I DAST: Client-Side (JavaScript) Cookie References 1

How to Fix: [Client-Side \(JavaScript\) Cookie References](#)

Issue 1 of 1

Issue ID:	3a47443e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/assets/index-cvw6f4ou.js
Domain	aaca.ntua.edu.tw
Element	const __vite__mapDeps=(i,m=__vite__mapDeps,d=(m.f (m.f=["assets/LoadSpecCss-DvpspM99.css","assets/L... (Page)
Path	/assets/index-cvw6f4ou.js
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, September 10, 2026
Last Updated	Thursday, September 10, 2026
CWE:	602

Issue 1 of 1 - Details

Reasoning: AppScan found a reference to cookies in the JavaScript.

Test Requests and Responses:

```

GET /assets/index-CVW6f4Ou.js HTTP/1.1
Host: aaca.ntua.edu.tw
Connection: keep-alive
Origin: https://aaca.ntua.edu.tw
sec-ch-ua-platform: "Linux"
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/145.0.0.0 Safari/537.36
sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
sec-ch-ua-mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: script
Referer: https://aaca.ntua.edu.tw/Server
Accept-Encoding: gzip
Accept-Language: en-US
Content-Length: 0

HTTP/1.1 200 OK
Cache-Control: public, max-age=31536000, immutable
Keep-Alive: timeout=5
Accept-Ranges: bytes
ETag: W/"17d4ab-1a085a42410"
Vary: Accept-Encoding
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=(), fullscreen=(self)

```


Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Cross-Origin-Resource-Policy: same-origin
X-Permitted-Cross-Domain-Policies: none
Strict-Transport-Security: max-age=16070400; includeSubDomains
Content-Security-Policy: default-src 'none'; require-trusted-types-for 'script'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'
X-Powered-By:
Date: Mon, 14 Sep 2026 02:42:25 GMT
Content-Type: text/javascript; charset=utf-8
Last-Modified: Wed, 09 Sep 2026 10:08:42 GMT
Content-Length: 1561771

```
const vite_mapDeps=(i,m=__vite_mapDeps,d=(m.f)||m.f["assets/LoadSpecCss-DvpspM99.css","assets/LoadSpecCss_Server-CvGY3VMi.css","assets/LoadFeaturesCss-DZPbrguC.js","assets/LoadFeaturesCss-D6S3uYhx.css","assets/cms_content_format-NvYllcok.css","assets/LoadFeaturesCss_Client-DC75pX19.js","assets/LoadFeaturesCss_Client-Cz6Vl-KF.css"])=>{i.map(i=>d[i]);function uL(e,t){for(var n=0;n<t.length;n++){const r=t[n];if(typeof r!="string"&&!Array.isArray(r)){for(const a in r){if(a!="default"&&(a in e)){const s=Object.getOwnPropertyDescriptor(r,a);s&&Object.defineProperty(e,a,s.get?{enumerable:!0,get:()=>r[a]}):}}return Object.freeze(Object.defineProperty(e,Symbol.toStringTag,{value:"Module"}))}}function(){const t=document.createElement("link").relList;if(t&&t.supports&&t.supports("modulepreload"))return;for(const a of document.querySelectorAll("link[rel='modulepreload']"))r(a);new MutationObserver(a=>{for(const s of a){if(s.type=="childList")for(const o of s.addedNodes)o.tagName=="LINK"&&o.rel=="modulepreload"&&r(o)}.observe(document,{childList:!0,subtree:!0});function n(a){const s={};return a.integrity&&(s.integrity=a.integrity),a.referrerPolicy&&(s.referrerPolicy=a.referrerPolicy),a.crossOrigin=="use-credentials"?s.credentials="include":a.crossOrigin=="anonymous"?s.credentials="omit":s.credentials="same-origin",s}function r(a){if(a.ep)return;a.ep=!0;const s=n(a);fetch(a.href,s)}const o5="modulepreload",l5=function(e){return"/"+e},CD={},sl=function(t,n,r){let a=Promise.resolve();if(n&&n.length>0){let o=function(f){return Promise.all(f.map(g=>Promise.resolve(g)).then(y=>({status:"fulfilled",value:y})),y=>({status:"rejected",reason:y})))};document.getElementsByTagName("link");const c=document.querySelector("meta[property=csp-nonce"]",d=c?c.get?c.getAttribute("nonce"):a=o(n.map(f=>{if(f=l5(f),f in CD)return;CD[f]=!0;const g=f.endsWith(".css"),y=g?[rel="stylesheet"]:"";if(document.querySelector(`link[href="${f}"]`))return;const p=document.createElement("link");if(p.rel=g?"stylesheet":o5,g){p.as="script",p.crossOrigin="",p.href=f,d&&p.setAttribute("nonce",d),document.head.appendChild(p)}return new Promise((w,b)=>{p.addEventListener("load",w),p.addEventListener("error",l)=>b(new Error("Unable to preload CSS for ${f}"))}})}function s(o){const c=new Event("vite:preloadError",{cancelable:!0});if(c.payload=o,window.dispatchEvent(c),!c.defaultPrevented)throw o}return a.then(o=>{for(const c of o){[c.status=="rejected"&&c.reason];return t().catch(s)}},function Cc(e){return e&&e._esModule&&Object.prototype.hasOwnProperty.call(e,"default"?e.default:e})var Mv={exports:{},tm:{},Ev={exports:{},kt:{}};/**  
 * @license React  
 * react.production.min.js  
 *  
 * Copyright (c) Facebook, Inc. and its affiliates.  
 *  
 * This source code is licensed under the MIT license found in the  
 * LICENSE file in the root directory of this source tree.  
 */var _D=function c5(){if(!_D)return kt;_D=1;var e=Symbol.for("react.element"),t=Symbol.for("react.portal"),n=Symbol.for("react.fragment"),r=Symbol.for("react.strict_mode"),a=Symbol.for("react.profiler"),s=Symbol.for("react.provider"),o=Symbol.for("react.context"),c=Symbol.for("react.forward_ref"),d=Symbol.for("react.suspense"),f=Symbol.for("react.memo"),g=Symbol.for("react.lazy"),y=Symbol.iterator;function p(A){return A===null||typeof A!="object"?null:(A=y&&A[y]||A["@@iterator"],typeof A=="function"?A:null)}var w={isMounted:function(){return!1},enqueueForceUpdate:function(){},enqueueReplaceState:function(){},enqueueSetState:function(){}},b=Object
```

[Go to Table of Contents](#)

I **DAST: Email Address Pattern Found** 4

How to Fix: [Email Address Pattern Found](#)

Issue 1 of 4

Issue ID:	a2ac433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/undergraduate-admissions/application-admission
Domain	aaca.ntua.edu.tw
Element	application-admission (Page)
Path	/undergraduate-admissions/application-admission
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	359

Issue 1 of 4 - Details

Reasoning: The response contains an e-mail address that may be private.

Test Requests and Responses:

```
GET /undergraduate-admissions/application-admission HTTP/1.1  
Host: aaca.ntua.edu.tw  
Connection: keep-alive  
Cache-Control: max-age=0  
sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"  
sec-ch-ua-mobile: ?0  
sec-ch-ua-platform: "Linux"
```


[illegible]

Issue 2 of 4

Issue ID:	27ac433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/
Domain	aaca.ntua.edu.tw
Path	/
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	359

Issue 2 of 4 - Details

Reasoning: The response contains an e-mail address that may be private.

Test Requests and Responses:

[illegible]

[illegible]

Issue 3 of 4

Issue ID:	6bad433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/news
Domain	aaca.ntua.edu.tw
Element	news (Page)
Path	/news
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	359

Issue 3 of 4 - Details

Reasoning: The response contains an e-mail address that may be private.

Test Requests and Responses:

```
GET /news HTTP/1.1
Host: aaca.ntua.edu.tw
Connection: keep-alive
Cache-Control: max-age=0
sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://aaca.ntua.edu.tw/Server/WebManagement/Announcement/List
Accept-Encoding: gzip
Accept-Language: en-US
Cookie: _Host-WCMS-Antiforgery=CfDj8HckeMFvsBBaitncu7Dg82yUciz_W6KLim7wXh_u1LCixd8xoFmHLisUUija51E-1oCoC0Sdt8Cfhmqaw5DvjEv5VbEa9zihoDJempfqXVLC--Eq19x0P0WwKaMijAedoRLYr8AtjdY2DpS7qLQXo; wcms.lang=zh-tw; access=eYJhbGQlIjUzIlNiIsInR5cCI6KpbXVCJ9.eYJlbNRClm5hbElkljoImMe1NzQ3Y2MtZTBjNy00MWZkLFwfMYtITOTNIYZuXNzcZNDY2IiwiaHR0cDovLzNjaGVtYXMueG1sc2h9cC5vcmcvd3MvMjAwNSw8NS9pZGVudGl0eS9jbGpfbXcvbmFtZWlkZkZlW50aWZpZXIIOiJhZG1pbjAyiLiwiHR0cDovLzNjaGVtYXMueG1sc2h9cC5vcmcvd3MvMjAwNSw8NS9pZGVudGl0eS9jbGpfbXcvbmFtZSI6ImFkbWluMDIlLCQqdGkiOiJNWZkyZAzoGNkZmE0MDAzYjE3MzYwNDE3NTFjMTk5NiIsIm5iZi16MTcxODM0OTMzMniwZXhwIjoxNzg5MzUxMTM2LCJpc3MiOiJXQ01TliwiYXVkljoivONNUy5BZGI1pbj9.mTAqrR4PXRAgUb6lE2oMrePO8ZCy4Nwwwvr7faOuqcPk; rtid=499776552abf8429fb1b4b35702f46acf; wcms.visitor=e2abdcd5c7ed44cbf8a1ebbbe4424a7022; XSRF-TOKEN=CfDj8HckeMFvsBBaitncu7Dg82x-QSa-KBNm1NnQwqlYRsEOBu5xH00hGogSPB_DBG5ELTF-9thMig_R6WUT0iOnVs0m7fhGxcXc_FSM5HZBeERjcldmCl55X9xqqWFtokvDjBc6g9xrww2piByMM_0t7HuAFVzmjh3uXi47T6tn50e6XhZ1QhdIHloAtDWwbfttu
```


[illegible]

Issue 4 of 4

Issue ID:	05ad433e-f36b-1410-89cd-00f37f576f24
Severity:	Informational
Status	Open
Location	https://aaca.ntua.edu.tw/undergraduate-admissions/four-year-and-two-year-college/537add47-65d9-442a-b074-cfb7e3a4d48f
Domain	aaca.ntua.edu.tw
Element	537add47-65d9-442a-b074-cfb7e3a4d48f (Page)
Path	/undergraduate-admissions/four-year-and-two-year-college/537add47-65d9-442a-b074-cfb7e3a4d48f
Scheme	https
Domain	aaca.ntua.edu.tw
CVSS	0.0
Date Created	Thursday, August 20, 2026
Last Updated	Thursday, August 20, 2026
CWE:	359

Issue 4 of 4 - Details

Reasoning: The response contains an e-mail address that may be private.

Test Requests and Responses:

```
GET /undergraduate-admissions/four-year-and-two-year-college/537add47-65d9-442a-b074-cfb7e3a4d48f HTTP/1.1
Accept-Language: en-US
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Referer: https://aaca.ntua.edu.tw/
Host: aaca.ntua.edu.tw
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36
Cookie: __Host-WCMS-Antiforgery=CfDJ8HckcMFvsBBaitncu7Dg82yUiciz_W6KLiM7wXh_u1LCxixd8xoFmHLisUIIja51E-1oCouCSD5t8Cfhmqw5DvjEvSvBEa9zihODJempfqXVLC-Eq19x0POWkRaMiJiaAedoRLYr8AtjdY2p57qLQXo; wcms.lang=zh-tw; access=eyljhbGciOiJIUzI1NiIsInR5cCI6IkpvcVJ9.eyllbnRlcm5hbElkloiMmE1NzQ3Y2MtZTBjNy00MWZKLWFmYTItOTNiYUxNzc2NDY2liwiaHR0cDovL3NjaGVtYXMuG1sc29hcC5vcmcvvd3MvYmJAwNS8wNS9pZGVudG0leS9ibGFpbXBvbmlFZSI6ImFkbWluMDIlLCJqdGkiOiJlNWZkYzAzOGNKZmEOMDAzyjE3MzYwZWY2NDY2NTFjMTk5NiIsIm5lZi6mTmC4OTM0OTMzNiwiZXhwIjozNg5MZUxMTM2MLTjc3MiOjIjXQ01TiwiYXVkllojV0NNUNU5BZC1pbjI9.mTAquR4PXaRuGub6IE2oMreP0BZCy4Nwwwr7faOuqcPk; rtid=49976652abf8429f91b4b35702f46acf; XSRF-TOKEN=CfDJ8HckcMFvsBBaitncu7Dg82xrppdRhZA-X365RBn8x4OpRuxV6p9f_wf_VrKj_79aOjiAErnXcfTP-YBsh7s796mpJpBOCxwiftfqoWMEYumJBzu95q_w3o9SU7vYAes7tqObR2toZYcojZ2A53kHlokbiQUoOPsIxotBYXuPy3V5jkM3a5_5qzwKVsoPvjw
Content-Length: 0
```

[illegible]

[Go to Table of Contents](#)

How to Fix

I Application Error

Cause

Proper bounds checking were not performed on incoming parameter values
No validation was done in order to make sure that user input matches the data type expected

Risk

It is possible to gather sensitive debugging information

If an attacker probes the application by forging a request that contains parameters or parameter values other than the ones expected by the application (examples are listed below), the application may enter an undefined state that makes it vulnerable to attack. The attacker can gain useful information from the application's response to this request, which information may be exploited to locate application weaknesses.

For example, if the parameter field should be an apostrophe-quoted string (e.g. in an ASP script or SQL query), the injected apostrophe symbol will prematurely terminate the string stream, thus changing the normal flow/syntax of the script.

Another cause of vital information being revealed in error messages, is when the scripting engine, web server, or database are misconfigured.

Here are some different variants:

- [1] Remove parameter
- [2] Remove parameter value
- [3] Set parameter value to null
- [4] Set parameter value to a numeric overflow (+/- 99999999)
- [5] Set parameter value to hazardous characters, such as ' " \' \ ") ;
- [6] Append some string to a numeric parameter value
- [7] Append "." (dot) or "[]" (angle brackets) to the parameter name

Fix recommendation

[1] Check incoming requests for the presence of all expected parameters and values. When a parameter is missing, issue a proper error message or use default values.

[2] The application should verify that its input consists of valid characters (after decoding). For example, an input value containing the null byte (encoded as %00), apostrophe, quotes, etc. should be rejected.

[3] Enforce values in their expected ranges and types. If your application expects a certain parameter to have a value from a certain set, then the application should ensure that the value it receives indeed belongs to the set. For example, if your application expects a value in the range 10..99, then it should make sure that the value is indeed numeric, and that its value is in 10..99.

[4] Verify that the data belongs to the set offered to the client.

[5] Do not output debugging error messages and exceptions in a production environment.

In order to disable debugging in ASP.NET, edit your web.config file to contain the following:

```
<compilation
debug="false"
/>
```

For more information, see "HOW TO: Disable Debugging for ASP.NET Applications" in:

<http://support.microsoft.com/default.aspx?scid=kb;en-us;815157>
■ <http://support.microsoft.com/default.aspx?scid=kb;en-us;815157>
external

You can add input validation to Web Forms pages by using validation controls. Validation controls provide an easy-to-use mechanism for all common types of standard validation (for example, testing for valid dates or values within a range), plus ways to provide custom-written validation. In addition, validation controls allow you to completely customize how error information is displayed to the user. Validation controls can be used with any controls that are processed in a Web Forms page's class file, including both HTML and Web server controls.

To make sure that all the required parameters exist in a request, use the "RequiredFieldValidator" validation control. This control ensures that the user does not skip an entry in the web form.

To make sure user input contains only valid values, you can use one of the following validation controls:

[1] "RangeValidator": checks that a user's entry (value) is between specified lower and upper boundaries. You can check ranges within pairs of numbers, alphabetic characters, and dates.

[2] "RegularExpressionValidator": checks that the entry matches a pattern defined by a regular expression. This type of validation allows you to check for predictable sequences of characters, such as those in social security numbers, e-mail addresses, telephone numbers, postal codes, and so on.

Important note: validation controls do not block user input or change the flow of page processing; they only set an error state, and produce error messages. It is the programmer's responsibility to test the state of the controls in the code before performing further application-specific actions.

There are two ways to check for user input validity:

1. Test for a general error state:

In your code, test the page's IsValid property. This property rolls up the values of the IsValid properties of all the validation controls on the page (using a logical AND). If one of the validation controls is set to invalid, the page's property will return false.

2. Test for the error state of individual controls:

Loop through the page's Validators collection, which contains references to all the validation controls. You can then examine the IsValid property of each validation control.

**** Input Data Validation:**

While data validations may be provided as a user convenience on the client-tier, data validation must be performed on the server-tier using Servlets. Client-side validations are inherently insecure because they can be easily bypassed, e.g. by disabling Javascript.

A good design usually requires the web application framework to provide server-side utility routines to validate the following:

[1] Required field

[2] Field data type (all HTTP request parameters are Strings by default)

[3] Field length

[4] Field range

[5] Field options

[6] Field pattern

[7] Cookie values

[8] HTTP Response

A good practice is to implement the above routine as static methods in a "Validator" utility class. The following sections describe an example validator class.

[1] Required field

Always check that the field is not null and its length is greater than zero, excluding leading and trailing white spaces.

Example of how to validate required fields:

```
// Java example to validate required fields public Class Validator { ... public static boolean validateRequired(String value)
{ boolean isValid = false; if (value != null && value.trim().length() > 0) { isValid = true; } return isValid; }
... } ... String fieldValue = request.getParameter("fieldName"); if (Validator.validateRequired(fieldValue)) { // fieldValue is
valid, continue processing request ... }
```

[2] Field data type

In web applications, input parameters are poorly typed. For example, all HTTP request parameters or cookie values are of type String. The developer is responsible for verifying the input is of the correct data type. Use the Java primitive wrapper classes to check if the field value can be safely converted to the desired primitive data type.

Example of how to validate a numeric field (type int):

```
// Java example to validate that a field is an int number public Class Validator { ... public static boolean validateInt(String
value) { boolean isValid = false; try { Integer.parseInt(value); isValid = true; } catch (Exception e) { isValid
```

```
= false; } return isFieldValid; } ... } ... // check if the HTTP request parameter is of type int String fieldValue =
request.getParameter("fieldName"); if (Validator.validateInt(fieldValue)) { // fieldValue is valid, continue processing
request ... }
```

A good practice is to convert all HTTP request parameters to their respective data types. For example, store the "integerValue" of a request parameter in a request attribute and use it as shown in the following example:

```
// Example to convert the HTTP request parameter to a primitive wrapper data type // and store this value in a request
attribute for further processing String fieldValue = request.getParameter("fieldName"); if
(Validator.validateInt(fieldValue)) { // convert fieldValue to an Integer Integer integerValue =
Integer.getInteger(fieldValue); // store integerValue in a request attribute request.setAttribute("fieldName",
integerValue); } ... // Use the request attribute for further processing Integer integerValue =
(Integer)request.getAttribute("fieldName"); ...
```

The primary Java data types that the application should handle:

- Byte
- Short
- Integer
- Long
- Float
- Double
- Date

[3] Field length

Always ensure that the input parameter (whether HTTP request parameter or cookie value) is bounded by a minimum length and/or a maximum length.

Example to validate that the length of the userName field is between 8 and 20 characters:

```
// Example to validate the field length public Class Validator { ... public static boolean validateLength(String value, int
minLength, int maxLength) { String validatedValue = value; if (!validateRequired(value)) { validatedValue = ""; } return
(validatedValue.length() >= minLength && validatedValue.length() <= maxLength); } ... } ... String userName =
request.getParameter("userName"); if (Validator.validateRequired(userName)) { if (Validator.validateLength(userName,
8, 20)) { // userName is valid, continue further processing ... } }
```

[4] Field range

Always ensure that the input parameter is within a range as defined by the functional requirements.

Example to validate that the input numberOfChoices is between 10 and 20:

```
// Example to validate the field range public Class Validator { ... public static boolean validateRange(int value, int min, int
max) { return (value >= min && value <= max); } ... } ... String fieldValue = request.getParameter("numberOfChoices");
if (Validator.validateRequired(fieldValue)) { if (Validator.validateInt(fieldValue)) { int numberOfChoices =
Integer.parseInt(fieldValue); if (Validator.validateRange(numberOfChoices, 10, 20)) { // numberOfChoices is valid,
continue processing request ... } } }
```

[5] Field options

Often, the web application presents the user with a set of options to choose from, e.g. using the SELECT HTML tag, but fails to perform server-side validation to ensure that the selected value is one of the allowed options. Remember that a malicious user can easily modify any option value. Always validate the selected user value against the allowed options as defined by the functional requirements.

Example to validate the user selection against a list of allowed options:

```
// Example to validate user selection against a list of options public Class Validator { ... public static boolean
validateOption(Object[] options, Object value) { boolean isValidValue = false; try { List list = Arrays.asList(options); if (list
!= null) { isValidValue = list.contains(value); } } catch (Exception e) { } return isValidValue; } ... } ... // Allowed options
String[] options = {"option1", "option2", "option3"}; // Verify that the user selection is one of the allowed options String
userSelection = request.getParameter("userSelection"); if (Validator.validateOption(options, userSelection)) { // valid
user selection, continue processing request ... }
```

[6] Field pattern

Always check that the user input matches a pattern as defined by the functionality requirements. For example, if the userName field should only allow alpha-numeric characters, case insensitive, then use the following regular expression:

```
^[a-zA-Z0-9]*$
```

Java 1.3 or earlier versions do not include any regular expression packages. Apache Regular Expression Package (see Resources below) is recommended for use with Java 1.3 to resolve this lack of support.

Example to perform regular expression validation:

```
// Example to validate that a given value matches a specified pattern // using the Apache regular expression package
import org.apache.regexp.RE; import org.apache.regexp.RESyntaxException; public Class Validator { ... public static
boolean matchPattern(String value, String expression) { boolean match = false; if (validateRequired(expression)) { RE r
= new RE(expression); match = r.match(value); } return match; } ... } ... // Verify that the userName request parameter
is alpha-numeric String userName = request.getParameter("userName"); if (Validator.matchPattern(userName, "^[a-zA-Z0-9]*$")) { // userName is valid, continue processing request ... }
```


Java 1.4 introduced a new regular expression package (java.util.regex). Here is a modified version of Validator.matchPattern using the new Java 1.4 regular expression package:

```
// Example to validate that a given value matches a specified pattern // using the Java 1.4 regular expression package
import java.util.regex.Pattern; import java.util.regex.Matcher; public Class Validator { ... public static boolean
matchPattern(String value, String expression) { boolean match = false; if (validateRequired(expression)) { match =
Pattern.matches(expression, value); } return match; } ... }
```

[7] Cookie value

Use the javax.servlet.http.Cookie object to validate the cookie value. The same validation rules (described above) apply to cookie values depending on the application requirements, e.g. validate a required value, validate length, etc.

Example to validate a required cookie value:

```
// Example to validate a required cookie value // First retrieve all available cookies submitted in the HTTP request Cookie[]
cookies = request.getCookies(); if (cookies != null) { // find the "user" cookie for (int i=0; i<cookies.length; ++i) { if
(cookies[i].getName().equals("user")) { // validate the cookie value if (Validator.validateRequired(cookies[i].getValue())) {
// valid cookie value, continue processing request ... } } }
```

[8] HTTP Response

[8-1] Filter user input

To guard the application against cross-site scripting, sanitize HTML by converting sensitive characters to their corresponding character entities. These are the HTML sensitive characters:

< > ' ' % ;) (& +

Example to filter a specified string by converting sensitive characters to their corresponding character entities:

```
// Example to filter sensitive data to prevent cross-site scripting public Class Validator { ... public static String filter(String
value) { if (value == null) { return null; } StringBuffer result = new StringBuffer(value.length()); for (int i=0;
i<value.length(); ++i) { switch (value.charAt(i)) { case '<': result.append("<"); break; case '>': result.append(">");
break; case '"': result.append("""); break; case '&': result.append("&"); break; case '%': result.append("%"); break; case ';':
result.append(";"); break; case '\': result.append("\'"); break; case '&': result.append("&"); break; case '&':
result.append("&"); break; case '+': result.append("+"); break; default: result.append(value.charAt(i)); break; } return
result; } ... } ... // Filter the HTTP response using Validator.filter PrintWriter out = response.getWriter(); // set output
response out.write(Validator.filter(response)); out.close();
```

The Java Servlet API 2.3 introduced Filters, which supports the interception and transformation of HTTP requests or responses.

Example of using a Servlet Filter to sanitize the response using Validator.filter:

```
// Example to filter all sensitive characters in the HTTP response using a Java Filter. // This example is for illustration
purposes since it will filter all content in the response, including HTML tags! public class SensitiveCharsFilter implements
Filter { ... public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws IOException,
ServletException { PrintWriter out = response.getWriter(); ResponseWrapper wrapper = new
ResponseWrapper((HttpServletResponse)response); chain.doFilter(request, wrapper); CharArrayWriter caw = new
CharArrayWriter(); caw.write(Validator.filter(wrapper.toString())); response.setContentType("text/html");
response.setContentLength(caw.toString().length()); out.write(caw.toString()); out.close(); } ... public class
CharResponseWrapper extends HttpServletResponseWrapper { private CharArrayWriter output; public String toString() {
return output.toString(); } public CharResponseWrapper(HttpServletResponse response){ super(response); output = new
CharArrayWriter(); } public PrintWriter getWriter(){ return new PrintWriter(output); } } }
```

[8-2] Secure the cookie

When storing sensitive data in a cookie, make sure to set the secure flag of the cookie in the HTTP response, using Cookie.setSecure(boolean flag) to instruct the browser to send the cookie using a secure protocol, such as HTTPS or SSL.

Example to secure the "user" cookie:

```
// Example to secure a cookie, i.e. instruct the browser to // send the cookie using a secure protocol Cookie cookie = new
Cookie("user", "sensitive"); cookie.setSecure(true); response.addCookie(cookie);
```

RECOMMENDED JAVA TOOLS

The two main Java frameworks for server-side validation are:

[1] Jakarta Commons Validator (integrated with Struts 1.1)

The Jakarta Commons Validator is a powerful framework that implements all the above data validation requirements. These rules are configured in an XML file that defines input validation rules for form fields. Struts supports output filtering of dangerous characters in the [8] HTTP Response by default on all data written using the Struts 'bean:write' tag. This filtering may be disabled by setting the 'filter=false' flag.

Struts defines the following basic input validators, but custom validators may also be defined:

required: succeeds if the field contains any characters other than white space.

mask: succeeds if the value matches the regular expression given by the mask attribute.

range: succeeds if the value is within the values given by the min and max attributes ((value >= min) & (value <= max)).

maxLength: succeeds if the field is length is less than or equal to the max attribute.

minLength: succeeds if the field is length is greater than or equal to the min attribute.

byte, short, integer, long, float, double: succeeds if the value can be converted to the corresponding primitive.

date: succeeds if the value represents a valid date. A date pattern may be provided.

creditCard: succeeds if the value could be a valid credit card number.

e-mail: succeeds if the value could be a valid e-mail address.

Example to validate the userName field of a loginForm using Struts Validator:

```
<form-validation> <global> ... <validator name="required" classname="org.apache.struts.validator.FieldChecks"
method="validateRequired" msg="errors.required"> </validator> <validator name="mask"
classname="org.apache.struts.validator.FieldChecks" method="validateMask" msg="errors.invalid"> </validator> ...
</global> <formset> <form name="loginForm"> <!-- userName is required and is alpha-numeric case insensitive -->
<field property="userName" depends="required,mask"> <!-- message resource key to display if validation fails -->
<msg name="mask" key="login.userName.maskmsg"/> <arg0 key="login.userName.displayName"/> <var> <var-
name>mask</var-name> <var-value>^[a-zA-Z0-9]*$</var-value> </var> </field> ... </form> ... </formset> </form-
validation>
```

[2] JavaServer Faces Technology

JavaServer Faces Technology is a set of Java APIs (JSR 127) to represent UI components, manage their state, handle events and input validation.

The JavaServer Faces API implements the following basic validators, but custom validators may be defined:

validate_doublerange: registers a DoubleRangeValidator on a component

validate_length: registers a LengthValidator on a component

validate_longrange: registers a LongRangeValidator on a component

validate_required: registers a RequiredValidator on a component

validate_stringrange: registers a StringRangeValidator on a component

validator: registers a custom Validator on a component

The JavaServer Faces API defines the following UIInput and UIOutput Renderers (Tags):

input_date: accepts a java.util.Date formatted with a java.text.Date instance

output_date: displays a java.util.Date formatted with a java.text.Date instance

input_datetime: accepts a java.util.Date formatted with a java.text.DateTime instance

output_datetime: displays a java.util.Date formatted with a java.text.DateTime instance

input_number: displays a numeric data type (java.lang.Number or primitive), formatted with a java.text.NumberFormat

output_number: displays a numeric data type (java.lang.Number or primitive), formatted with a java.text.NumberFormat

input_text: accepts a text string of one line.

output_text: displays a text string of one line.

input_time: accepts a java.util.Date, formatted with a java.text.DateFormat time instance

output_time: displays a java.util.Date, formatted with a java.text.DateFormat time instance

input_hidden: allows a page author to include a hidden variable in a page

input_secret: accepts one line of text with no spaces and displays it as a set of asterisks as it is typed

input_textarea: accepts multiple lines of text

output_errors: displays error messages for an entire page or error messages associated with a specified client identifier

output_label: displays a nested component as a label for a specified input field

output_message: displays a localized message

Example to validate the userName field of a loginForm using JavaServer Faces:

```
<%@ taglib uri="https://docs.oracle.com/javaee/6/tutorial/doc/glxce.html" prefix="h" %> <%@ taglib
uri="http://mrbool.com/how-to-create-a-login-validation-with-jsf-java-server-faces/27046" prefix="f" %> ... <jsp:useBean
id="UserBean" class="myApplication.UserBean" scope="session" /> <f:use_faces> <h:form formName="loginForm" >
<h:input_text id="userName" size="20" modelReference="UserBean.userName"> <f:validate_required/>
<f:validate_length minimum="8" maximum="20"/> </h:input_text> <!-- display errors if present --> <h:output_errors
id="loginErrors" clientId="userName"/> <h:command_button id="submit" label="Submit" commandName="submit" />
<p> </h:form> </f:use_faces>
```

REFERENCES

Java API 1.3 -

<https://www.oracle.com/java/technologies/java-archive-13docs-downloads.html>

■ <https://www.oracle.com/java/technologies/java-archive-13docs-downloads.html>
external

Java API 1.4 -

<https://www.oracle.com/java/technologies/java-archive-142docs-downloads.html>
■ <https://www.oracle.com/java/technologies/java-archive-142docs-downloads.html>
external

Java Servlet API 2.3 -

<https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api>
■ <https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api>
external

Java Regular Expression Package -

<http://jakarta.apache.org/regexp/>
■ <http://jakarta.apache.org/regexp/>
external

Jakarta Validator -

<http://jakarta.apache.org/commons/validator/>
■ <http://jakarta.apache.org/commons/validator/>
external

JavaServer Faces Technology -

<http://www.java-serverfaces.org/>
■ <http://www.java-serverfaces.org/>
external

**** Error Handling:**

Many J2EE web application architectures follow the Model View Controller (MVC) pattern. In this pattern a Servlet acts as a Controller. A Servlet delegates the application processing to a JavaBean such as an EJB Session Bean (the Model). The Servlet then forwards the request to a JSP (View) to render the processing results. Servlets should check all input, output, return codes, error codes and known exceptions to ensure that the expected processing actually occurred.

While data validation protects applications against malicious data tampering, a sound error handling strategy is necessary to prevent the application from inadvertently disclosing internal error messages such as exception stack traces. A good error handling strategy addresses the following items:

[1] Defining Errors

[2] Reporting Errors

[3] Rendering Errors

[4] Error Mapping

[1] Defining Errors

Hard-coded error messages in the application layer (e.g. Servlets) should be avoided. Instead, the application should use error keys that map to known application failures. A good practice is to define error keys that map to validation rules for HTML form fields or other bean properties. For example, if the "user_name" field is required, is alphanumeric, and must be unique in the database, then the following error keys should be defined:

(a) ERROR_USERNAME_REQUIRED: this error key is used to display a message notifying the user that the "user_name" field is required;

(b) ERROR_USERNAME_ALPHANUMERIC: this error key is used to display a message notifying the user that the "user_name" field should be alphanumeric;

(c) ERROR_USERNAME_DUPLICATE: this error key is used to display a message notifying the user that the "user_name" value is a duplicate in the database;

(d) ERROR_USERNAME_INVALID: this error key is used to display a generic message notifying the user that the "user_name" value is invalid;

A good practice is to define the following framework Java classes which are used to store and report application errors:

- ErrorKeys: defines all error keys

```
// Example: ErrorKeys defining the following error keys: // - ERROR_USERNAME_REQUIRED // -  
ERROR_USERNAME_ALPHANUMERIC // - ERROR_USERNAME_DUPLICATE // - ERROR_USERNAME_INVALID // ... public Class  
ErrorKeys { public static final String ERROR_USERNAME_REQUIRED = "error.username.required"; public static final String  
ERROR_USERNAME_ALPHANUMERIC = "error.username.alphanumeric"; public static final String  
ERROR_USERNAME_DUPLICATE = "error.username.duplicate"; public static final String ERROR_USERNAME_INVALID =  
"error.username.invalid"; ... }
```

- Error: encapsulates an individual error

```
// Example: Error encapsulates an error key. // Error is serializable to support code executing in multiple JVMs. public  
Class Error implements Serializable { // Constructor given a specified error key public Error(String key) { this(key, null); }  
// Constructor given a specified error key and array of placeholder objects public Error(String key, Object[] values) {  
this.key = key; this.values = values; } // Returns the error key public String getKey() { return this.key; } // Returns the  
placeholder values public Object[] getValues() { return this.values; } private String key = null; private Object[] values =  
null; }
```

- Errors: encapsulates a Collection of errors

```
// Example: Errors encapsulates the Error objects being reported to the presentation layer. // Errors are stored in a
HashMap where the key is the bean property name and value is an // ArrayList of Error objects. public Class Errors
implements Serializable { // Adds an Error object to the Collection of errors for the specified bean property. public void
addError(String property, Error error) { ArrayList propertyErrors = (ArrayList)errors.get(property); if (propertyErrors ==
null) { propertyErrors = new ArrayList(); errors.put(property, propertyErrors); } propertyErrors.put(error); } // Returns
true if there are any errors public boolean hasErrors() { return (errors.size > 0); } // Returns the Errors for the specified
property public ArrayList getErrors(String property) { return (ArrayList)errors.get(property); } private HashMap errors =
new HashMap(); }
```

Using the above framework classes, here is an example to process validation errors of the "user_name" field:

```
// Example to process validation errors of the "user_name" field. Errors errors = new Errors(); String userName =
request.getParameter("user_name"); // (a) Required validation rule if (!Validator.validateRequired(userName)) {
errors.addError("user_name", new Error(ErrorKeys.ERROR_USERNAME_REQUIRED)); } // (b) Alpha-numeric validation rule
else if (!Validator.matchPattern(userName, "^[a-zA-Z0-9]*$")) { errors.addError("user_name", new
Error(ErrorKeys.ERROR_USERNAME_ALPHANUMERIC)); } else { // (c) Duplicate check validation rule // We assume that
there is an existing UserValidationEJB session bean that implements // a checkIfDuplicate() method to verify if the user
already exists in the database. try { ... if (UserValidationEJB.checkIfDuplicate(userName)) { errors.addError("user_name",
new Error(ErrorKeys.ERROR_USERNAME_DUPLICATE)); } } catch (RemoteException e) { // log the error
logger.error("Could not validate user for specified userName: " + userName); errors.addError("user_name", new
Error(ErrorKeys.ERROR_USERNAME_DUPLICATE)); } } // set the errors object in a request attribute called "errors"
request.setAttribute("errors", errors); ...
```

[2] Reporting Errors

There are two ways to report web-tier application errors:

(a) Servlet Error Mechanism

(b) JSP Error Mechanism

[2-a] Servlet Error Mechanism

A Servlet may report errors by:

- forwarding to the input JSP (having already stored the errors in a request attribute), OR
- calling response.sendError with an HTTP error code argument, OR
- throwing an exception

It is good practice to process all known application errors (as described in section [1]), store them in a request attribute, and forward to the input JSP. The input JSP should display the error messages and prompt the user to re-enter the data. The following example illustrates how to forward to an input JSP (userInput.jsp):

```
// Example to forward to the userInput.jsp following user validation errors RequestDispatcher rd =
getServletContext().getRequestDispatcher("/user/userInput.jsp"); if (rd != null) { rd.forward(request, response); }
```

If the Servlet cannot forward to a known JSP page, the second option is to report an error using the response.sendError method with HttpServletResponse.SC_INTERNAL_SERVER_ERROR (status code 500) as argument. Refer to the javadoc of javax.servlet.http.HttpServletResponse for more details on the various HTTP status codes.

Example to return a HTTP error:

```
// Example to return a HTTP error code RequestDispatcher rd =
getServletContext().getRequestDispatcher("/user/userInput.jsp"); if (rd == null) { // messages is a resource bundle with
all message keys and values response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
messages.getMessage(ErrorKeys.ERROR_USERNAME_INVALID)); }
```

As a last resort, Servlets can throw an exception, which must be a subclass of one of the following classes:

- RuntimeException
- ServletException
- IOException

[2-b] JSP Error Mechanism

JSP pages provide a mechanism to handle runtime exceptions by defining an errorPage directive as shown in the following example:

```
<%@ page errorPage="/errors/userValidation.jsp" %>
```

Uncaught JSP exceptions are forwarded to the specified errorPage, and the original exception is set in a request parameter called javax.servlet.jsp.jspException. The error page must include a isErrorPage directive as shown below:

```
<%@ page isErrorPage="true" %>
```

The isErrorPage directive causes the "exception" variable to be initialized to the exception object being thrown.

[3] Rendering Errors

The J2SE Internationalization APIs provide utility classes for externalizing application resources and formatting messages including:

- (a) Resource Bundles
- (b) Message Formatting

[3-a] Resource Bundles

Resource bundles support internationalization by separating localized data from the source code that uses it. Each resource bundle stores a map of key/value pairs for a specific locale.

It is common to use or extend `java.util.PropertyResourceBundle`, which stores the content in an external properties file as shown in the following example:

```
##### # ErrorMessages.properties
##### # required user name error message
error.username.required=User name field is required # invalid user name format error.username.alphanumeric=User
name must be alphanumeric # duplicate user name error message error.username.duplicate=User name {0} already
exists, please choose another one ...
```

Multiple resources can be defined to support different locales (hence the name resource bundle). For example, `ErrorMessages_fr.properties` can be defined to support the French member of the bundle family. If the resource member of the requested locale does not exist, the default member is used. In the above example, the default resource is `ErrorMessages.properties`. Depending on the user's locale, the application (JSP or Servlet) retrieves content from the appropriate resource.

[3-b] Message Formatting

The J2SE standard class `java.util.MessageFormat` provides a generic way to create messages with replacement placeholders. A `MessageFormat` object contains a pattern string with embedded format specifiers as shown below:

```
// Example to show how to format a message using placeholder parameters String pattern = "User name {0} already
exists, please choose another one"; String userName = request.getParameter("user_name"); Object[] args = new
Object[1]; args[0] = userName; String message = MessageFormat.format(pattern, args);
```

Here is a more comprehensive example to render error messages using `ResourceBundle` and `MessageFormat`:

```
// Example to render an error message from a localized ErrorMessages resource (properties file) // Utility class to retrieve
locale-specific error messages public Class ErrorMessageResource { // Returns the error message for the specified error
key in the environment locale public String getErrorMessage(String errorKey) { return getErrorMessage(errorKey,
defaultLocale); } // Returns the error message for the specified error key in the specified locale public String
getErrorMessage(String errorKey, Locale locale) { return getErrorMessage(errorKey, null, locale); } // Returns a formatted
error message for the specified error key in the specified locale public String getErrorMessage(String errorKey, Object[]
args, Locale locale) { // Get localized ErrorMessageResource ResourceBundle errorMessageResource =
ResourceBundle.getBundle("ErrorMessages", locale); // Get localized error message String errorMessage =
errorMessageResource.getString(errorKey); if (args != null) { // Format the message using the specified placeholders
args return MessageFormat.format(errorMessage, args); } else { return errorMessage; } } // default environment locale
private Locale defaultLocale = Locale.getDefaultLocale(); } ... // Get the user's locale Locale userLocale =
request.getLocale(); // Check if there were any validation errors Errors errors = (Errors)request.getAttribute("errors"); if
(errors != null && errors.hasErrors()) { // iterate through errors and output error messages corresponding to the
"user_name" property ArrayList userNameErrors = errors.getErrors("user_name"); ListIterator iterator =
userNameErrors.iterator(); while (iterator.hasNext()) { // Get the next error object Error error = (Error)iterator.next();
String errorMessage = ErrorMessageResource.getErrorMessage(error.getKey(), userLocale); output.write(errorMessage +
"\r\n"); } }
```

It is recommended to define a custom JSP tag, e.g. `displayErrors`, to iterate through and render error messages as shown in the above example.

[4] Error Mapping

Normally, the Servlet Container will return a default error page corresponding to either the response status code or the exception. A mapping between the status code or the exception and a web resource may be specified using custom error pages. It is a good practice to develop static error pages that do not disclose internal error states (by default, most Servlet containers will report internal error messages). This mapping is configured in the Web Deployment Descriptor (`web.xml`) as specified in the following example:

```
<!-- Mapping of HTTP error codes and application exceptions to error pages --> <error-page> <exception-
type>UserValidationException</exception-type> <location>/errors/validationError.html</error-page> </error-page>
<error-page> <error-code>500</error-code> <exception-type> <location>/errors/internalError.html</error-page> </error-page>
<error-page> ... </error-page> ...
```

RECOMMENDED JAVA TOOLS

The two main Java frameworks for server-side validation are:

[1] Jakarta Commons Validator (integrated with Struts 1.1)

The Jakarta Commons Validator is a Java framework that defines the error handling mechanism as described above. Validation rules are configured in an XML file that defines input validation rules for form fields and the corresponding validation error keys. Struts provides internationalization support to build localized applications using resource bundles and message formatting.

Example to validate the `userName` field of a loginForm using Struts Validator:

```
<form-validation> <global> ... <validator name="required" classname="org.apache.struts.validator.FieldChecks"
method="validateRequired" msg="errors.required"> </validator> <validator name="mask"
classname="org.apache.struts.validator.FieldChecks" method="validateMask" msg="errors.invalid"> </validator> ...
</global> <formset> <form name="loginForm"> <!-- userName is required and is alpha-numeric case insensitive -->
<field property="userName" depends="required,mask"> <!-- message resource key to display if validation fails -->
<msg name="mask" key="login.userName.maskmsg"/> <arg0 key="login.userName.displayName"/> <var> <var-
name>mask</var-name> <var-value>^[a-zA-Z0-9]*$</var-value> </var> </field> ... </form> ... </formset> </form-
validation>
```

The Struts JSP tag library defines the `"errors"` tag that conditionally displays a set of accumulated error messages as shown in the following example:

```
<%@ page language="java" %> <%@ taglib uri="/WEB-INF/struts-html.tld" prefix="html" %> <%@ taglib uri="/WEB-INF/struts-bean.tld" prefix="bean" %> <html:html> <head> <body> <html:form action="/login.do"> <table border="0" width="100%"> <tr> <th align="right"> <html:errors property="username"/> <bean:message key="prompt.username"/> </th> <td align="left"> <html:text property="username" size="16"/> </td> </tr> <tr> <td align="right"> <html:submit><bean:message key="button.submit"/></html:submit> </td> <td align="right"> <html:reset><bean:message key="button.reset"/></html:reset> </td> </tr> </table> </html:form> </body> </html:html>
```

[2] JavaServer Faces Technology

JavaServer Faces Technology is a set of Java APIs (JSR 127) to represent UI components, manage their state, handle events, validate input, and support internationalization.

The JavaServer Faces API defines the "output_errors" UIOutput Renderer, which displays error messages for an entire page or error messages associated with a specified client identifier.

Example to validate the userName field of a loginForm using JavaServer Faces:

```
<%@ taglib uri="https://docs.oracle.com/javaee/6/tutorial/doc/glxce.html" prefix="h" %> <%@ taglib uri="http://mrbool.com/how-to-create-a-login-validation-with-jsf-java-server-faces/27046" prefix="f" %> ... <jsp:useBean id="UserBean" class="myApplication.UserBean" scope="session" /> <f:use_faces> <h:form formName="loginForm" > <h:input_text id="userName" size="20" modelReference="UserBean.userName"> <f:validate_required/> <f:validate_length minimum="8" maximum="20"/> </h:input_text> <!-- display errors if present --> <h:output_errors id="loginErrors" clientId="userName"/> <h:command_button id="submit" label="Submit" commandName="submit" /> <p> </h:form> </f:use_faces>
```

REFERENCES

Java API 1.3 -

<https://www.oracle.com/java/technologies/java-archive-13docs-downloads.html>
 ■ <https://www.oracle.com/java/technologies/java-archive-13docs-downloads.html>
 external

Java API 1.4 -

<https://www.oracle.com/java/technologies/java-archive-142docs-downloads.html>
 ■ <https://www.oracle.com/java/technologies/java-archive-142docs-downloads.html>
 external

Java Servlet API 2.3 -

<https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api>
 ■ <https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api>
 external

Java Regular Expression Package -

<http://jakarta.apache.org/regexp/>
 ■ <http://jakarta.apache.org/regexp/>
 external

Jakarta Validator -

<http://jakarta.apache.org/commons/validator/>
 ■ <http://jakarta.apache.org/commons/validator/>
 external

JavaServer Faces Technology -

<http://www.javaserverfaces.org/>
 ■ <http://www.javaserverfaces.org/>
 external

** Input Data Validation:

While data validations may be provided as a user convenience on the client-tier, data validation must always be performed on the server-tier. Client-side validations are inherently insecure because they can be easily bypassed, e.g. by disabling Javascript.

A good design usually requires the web application framework to provide server-side utility routines to validate the following:

- [1] Required field
- [2] Field data type (all HTTP request parameters are Strings by default)
- [3] Field length
- [4] Field range
- [5] Field options
- [6] Field pattern
- [7] Cookie values
- [8] HTTP Response

A good practice is to implement a function or functions that validates each application parameter. The following sections

describe some example checking.

[1] Required field

Always check that the field is not null and its length is greater than zero, excluding leading and trailing white spaces.

Example of how to validate required fields:

```
// PHP example to validate required fields function validateRequired($input) { ... $pass = false; if (strlen(trim($input))>0) { $pass = true; } return $pass; ... } ... if (validateRequired($fieldName)) { // fieldName is valid, continue processing request ... }
```

[2] Field data type

In web applications, input parameters are poorly typed. For example, all HTTP request parameters or cookie values are of type String. The developer is responsible for verifying the input is of the correct data type.

[3] Field length

Always ensure that the input parameter (whether HTTP request parameter or cookie value) is bounded by a minimum length and/or a maximum length.

[4] Field range

Always ensure that the input parameter is within a range as defined by the functional requirements.

[5] Field options

Often, the web application presents the user with a set of options to choose from, e.g. using the SELECT HTML tag, but fails to perform server-side validation to ensure that the selected value is one of the allowed options. Remember that a malicious user can easily modify any option value. Always validate the selected user value against the allowed options as defined by the functional requirements.

[6] Field pattern

Always check that user input matches a pattern as defined by the functionality requirements. For example, if the userName field should only allow alpha-numeric characters, case insensitive, then use the following regular expression:

```
^[a-zA-Z0-9]+$
```

[7] Cookie value

The same validation rules (described above) apply to cookie values depending on the application requirements, e.g. validate a required value, validate length, etc.

[8] HTTP Response

[8-1] Filter user input

To guard the application against cross-site scripting, the developer should sanitize HTML by converting sensitive characters to their corresponding character entities. These are the HTML sensitive characters:

```
< > " ' % ; ) ( & +
```

PHP includes some automatic sanitization utility functions, such as htmlentities():

```
$input = htmlentities($input, ENT_QUOTES, 'UTF-8');
```

In addition, in order to avoid UTF-7 variants of Cross-site Scripting, you should explicitly define the Content-Type header of the response, for example:

```
<?php header('Content-Type: text/html; charset=UTF-8'); ?>
```

[8-2] Secure the cookie

When storing sensitive data in a cookie and transporting it over SSL, make sure that you first set the secure flag of the cookie in the HTTP response. This will instruct the browser to only use that cookie over SSL connections.

You can use the following code example, for securing the cookie:

```
<$php $value = "some_value"; $time = time()+3600; $path = "/application/"; $domain = ".example.com"; $secure = 1; setcookie("CookieName", $value, $time, $path, $domain, $secure, TRUE); ?>
```

In addition, we recommend that you use the HttpOnly flag. When the HttpOnly flag is set to TRUE the cookie will be made accessible only through the HTTP protocol. This means that the cookie won't be accessible by scripting languages, such as JavaScript. This setting can effectively help to reduce identity theft through XSS attacks (although it is not supported by all browsers).

The HttpOnly flag was Added in PHP 5.2.0.

REFERENCES

[1] Mitigating Cross-site Scripting With HTTP-only Cookies:

<http://msdn2.microsoft.com/en-us/library/ms533046.aspx>
■ <http://msdn2.microsoft.com/en-us/library/ms533046.aspx>
external

[2] PHP Security Consortium:

<http://phpsec.org/>
▪ <http://phpsec.org/>
external

[3] PHP & Web Application Security Blog (Chris Shiflett):

<http://shiflett.org/>
▪ <http://shiflett.org/>
external

CWE

550

External references

- [An example for using apostrophe to hack a site can be found in "How I hacked PacketStorm \(by Rain Forest Puppy\), RFP's site"](#)
- ["Web Application Disassembly with ODBC Error Messages" \(By David Litchfield\)](#)
- [CERT Advisory \(CA-1997-25\): Sanitizing user-supplied data in CGI scripts](#)

[Go to Table of Contents](#)

I Client-Side (JavaScript) Cookie References

Cause

Cookies are created at the client side

Risk

The worst case scenario for this attack depends on the context and role of the cookies that are created at the client side

A cookie is a piece of information usually created by the Web server and stored in the Web browser.

The cookie contains information used by web applications mainly (but not only) to identify users and maintain their state.

AppScan detected that the JavaScript code at the client side is used to manipulate (either create or modify) the site's cookies.

It is possible for an attacker to view this code, understand its logic, and use it to compose his own cookies, or modify existing ones, based on this knowledge.

The damage an attacker may cause depends on how the application uses its cookies, or what information it stores in them.

Among other things, cookie manipulation may lead to session hijacking or privilege escalation.

Other vulnerabilities caused by cookie poisoning contain SQL injection and Cross-Site scripting.

Fix recommendation

[1] Avoid placing business/security logic at the client side.

[2] Find and remove insecure client-side Javascript code which may pose a security threat to the site.

CWE

602

External references

- [WASC Threat Classification: Information Leakage](#)

[Go to Table of Contents](#)

Cause

Sensitive Cookie with Improper or Insecure or Missing SameSite Attribute

Risk

Prevent Cookie information leakage by restricting cookies to first-party or same-site context

Attacks can extend to Cross-Site-Request-Forgery (CSRF) attacks if there are no additional protections in place (such as Anti-CSRF tokens).

The SameSite attribute controls how cookies are sent for cross-domain requests.

The attribute may have three values: 'Lax', 'Strict', or 'None'. If 'None' is used, a website may create a cross-domain POST HTTP request to another website, and the browser automatically adds cookies to this request.

This may lead to Cross-Site-Request-Forgery (CSRF) attacks if there are no additional protections in place (such as Anti-CSRF tokens).

Modes and their uses:

'Lax' mode: the cookie will only be sent with a top-level get request.

'Strict' mode; the cookie will not be sent with any cross-site usage even if the user follows a link to another website.

'None' mode: the cookie will be sent with the cross-site requests.

The attribute having: 'Lax' or 'None' must have 'Secure' Flag set and must be transferred over https.

Example - Set-Cookie: key=value; SameSite=Lax;Secure

Setting attribute to 'Strict' is the recommended option.

Example - Set-Cookie: key=value; SameSite=Strict

Fix recommendation

[1] Review possible solutions for configuring SameSite Cookie attribute to recommended values.

[2] Restrict Cookies to a first-party or same-site context.

[3] Verify and set the SameSite attribute of your cookie to Strict, to ensure that the cookie will only be sent in a first-party context.

[4] Or, if you want to relax the restrictions of first-party context, then verify and set the SameSite attribute of the cookie to Lax with Secure Flag enabled and transferred over HTTPS.

CWE

1275

External references

- [WASC Threat Classification: Information Leakage](#)
- [SameSite Cookies](#)

[Go to Table of Contents](#)

Cause

Insecure web application programming or configuration

Risk

It is possible to gather sensitive information about the web application such as usernames, passwords, machine name and/or sensitive file locations

Spambots crawl internet sites, set out to find e-mail addresses in order to build mailing lists for sending unsolicited e-mail (spam).

AppScan detected a response containing one or more e-mail addresses, which may be exploited to send spam mail

Furthermore, the e-mail addresses found may be private and thus should not be accessible to the general public.

Fix recommendation

Remove any e-mail addresses from the website so that they won't be exploited by malicious users.

CWE

359

External references

- [Definition of Spambot \(Wikipedia\)](#)

[Go to Table of Contents](#)

L Trusted types in scripts not enforced in CSP

Cause

Insecure web application programming or configuration

Risk

The "Content-Security-Policy" header is designed to modify the way browsers render pages, and thus to protect from various cross-site injections, including Cross-Site Scripting. It is important to set the header value correctly, in a way that will not prevent proper operation of the web site.

Not using Trusted Types can allow attackers to inject scripts, as a result, this increases the risk of XSS vulnerabilities.

Fix recommendation

Consider using 'require-trusted-types-for' directive to prevent DOM-based XSS attacks.

Add "require-trusted-types-for 'script'" directive to your CSP to enforce Trusted Types for scripts.

CWE

1032

External references

- [MDN web docs - Content-Security-Policy](#)

[Go to Table of Contents](#)